BINARY SEARCH TREES

Problem Solving with Computers-II

Join iclicker at **https://join.iclicker.com/GEXG**

Problem Scenario: Building Leetcode

You are building a competitive coding platform (think LeetCode contests). During a live contest, three things happen constantly:

- A student submits a solution and their **score updates** (insert/update)
- The UI needs to display the leaderboard in ranked order at any time
- A student wants to know "who is ranked just above and below me?" (predecessor/successor)

Discuss: Why does a sorted array of scores fall short in this case?

Problem Scenario: Task Scheduler

Your laptop runs Spotify, Chrome, and a virus scan simultaneously
The CPU can only run one task at a time.

Rule: run whichever task has received the least CPU time.

Thousands of times per second:

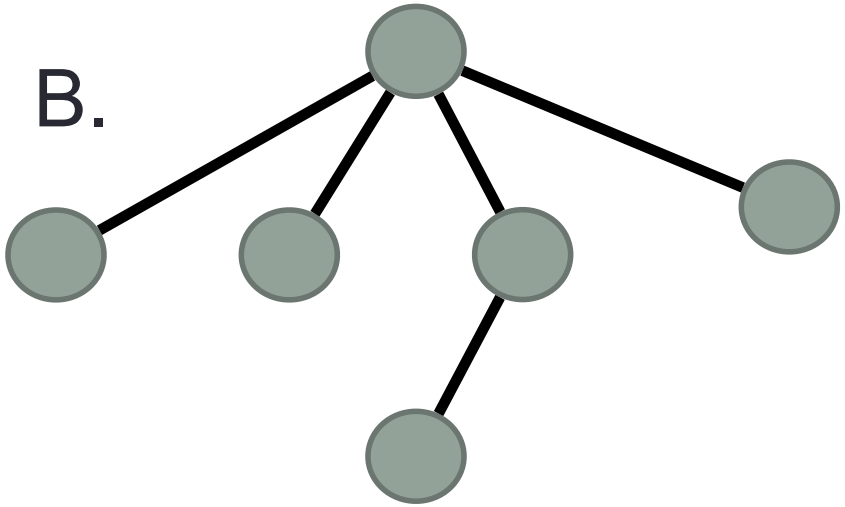
- Find the most starved task
- Run it briefly
- Update its total, insert it back

Discuss: Why does a sorted array fail when a task's CPU time usage changes every few milliseconds?

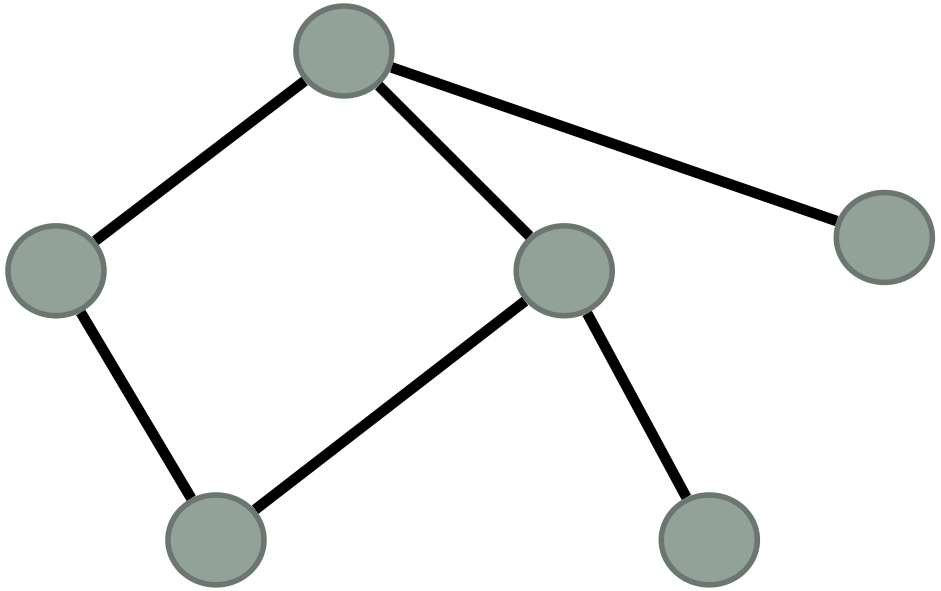
Which of the following is/are a tree?

A.

B.



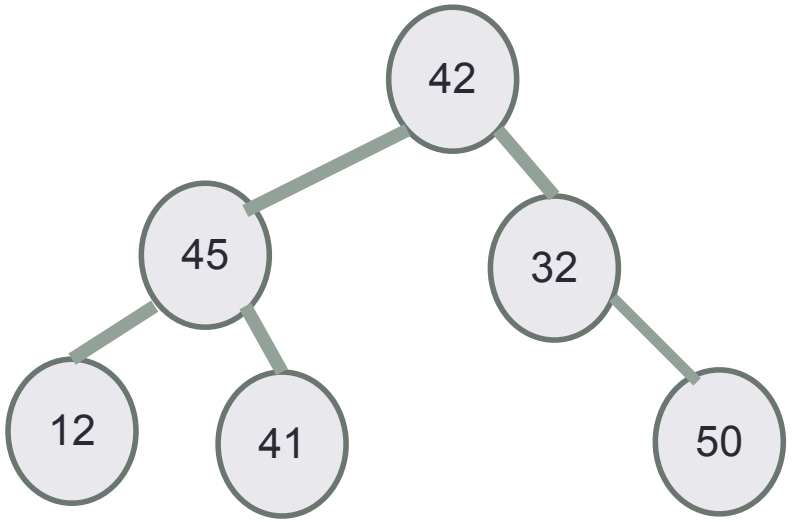
C.



D. A & B

E. All of A-C

Binary Trees



In a tree, nodes are arranged in a hierarchy

- One node is distinguished as the root
- Each node:
  - stores a key
  - has a pointer to child nodes and parent (optional)
- Unique path between any two nodes
- Leaf nodes have no children

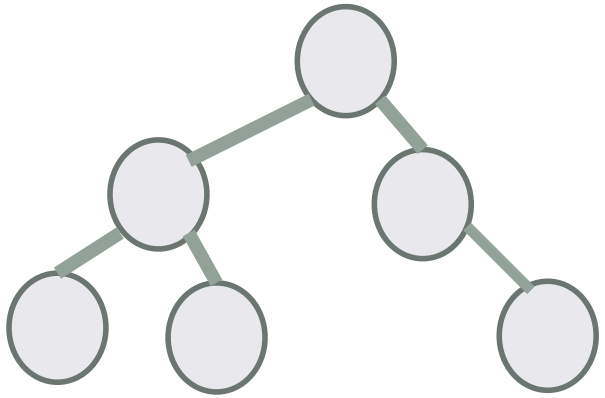
In a binary tree, each node has at most two children

```
struct TreeNode {

  TreeNode* left;
  TreeNode* right;
  TreeNode* parent;
  int const data;

  TreeNode(int d) : data(d) {
    left = right = parent = nullptr;
  }
};
```

Binary Search Tree – What is it?



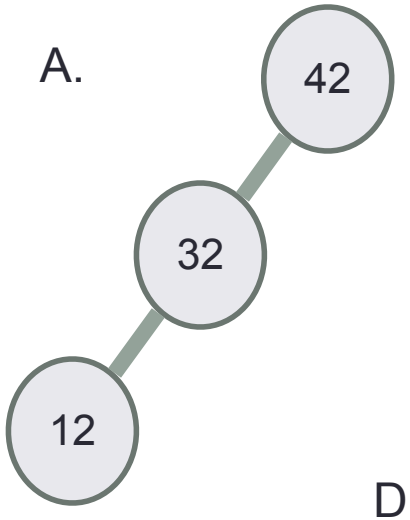
BST is a binary tree where each node satisfies the Search Tree Property

Do the keys have to be integers?

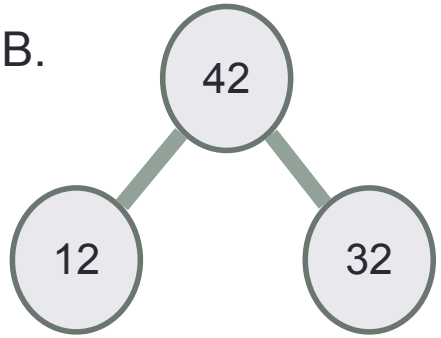
Visualization: https://visualgo.net/en/bst

Which of the following is/are a binary search tree?

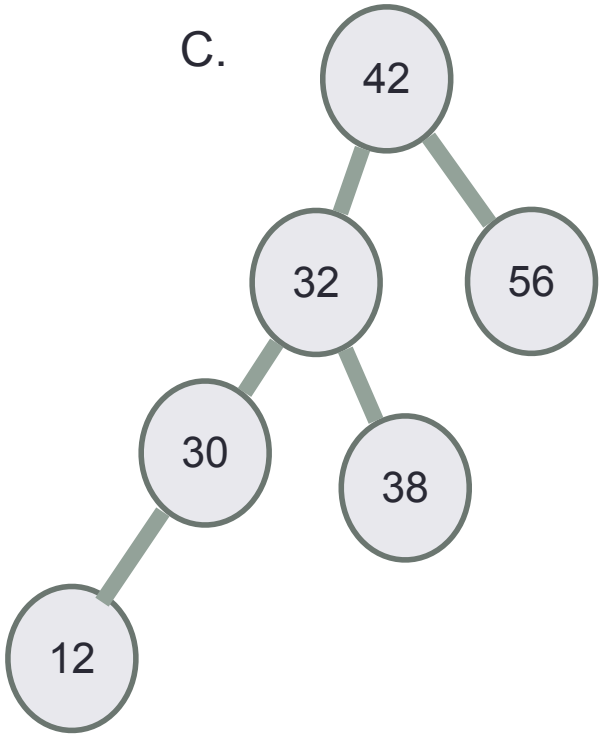
A.



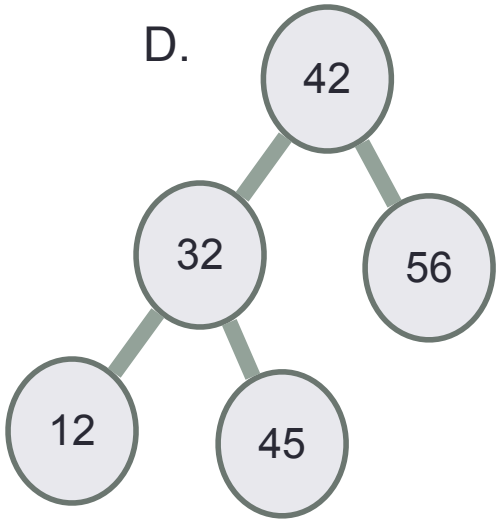
B.



C.

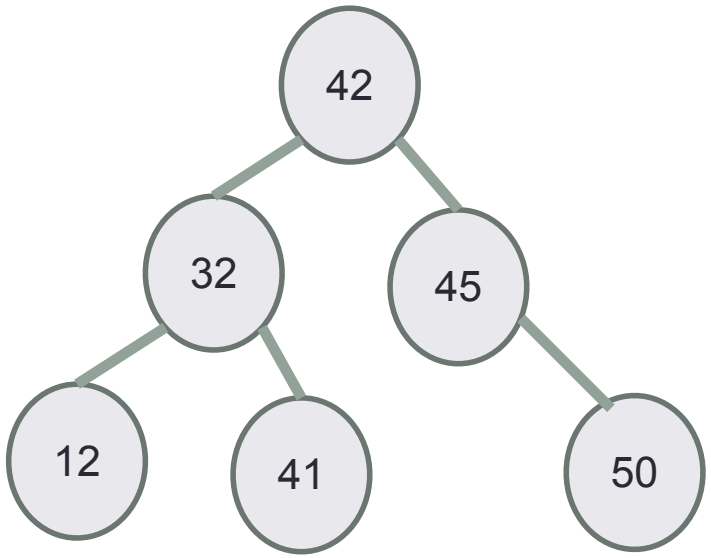


D.



E. More than one of these

Search



- Start at the root;
- Trace down a path by comparing **k** with the key of the current node x:
    - If the keys are equal: we have found the key
    - If **k** < key[x] search in the left subtree of x
    - If **k** > key[x] search in the right subtree of x
- What is the running time of search?

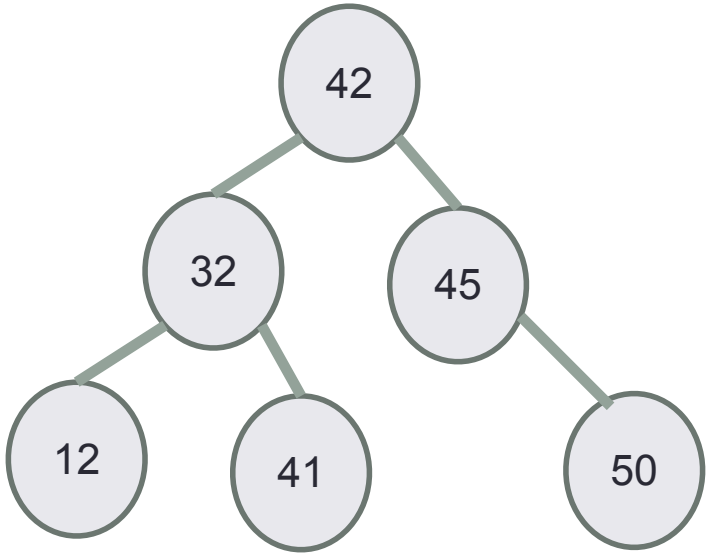
Search for 41, then search for 53

Operation: Insert

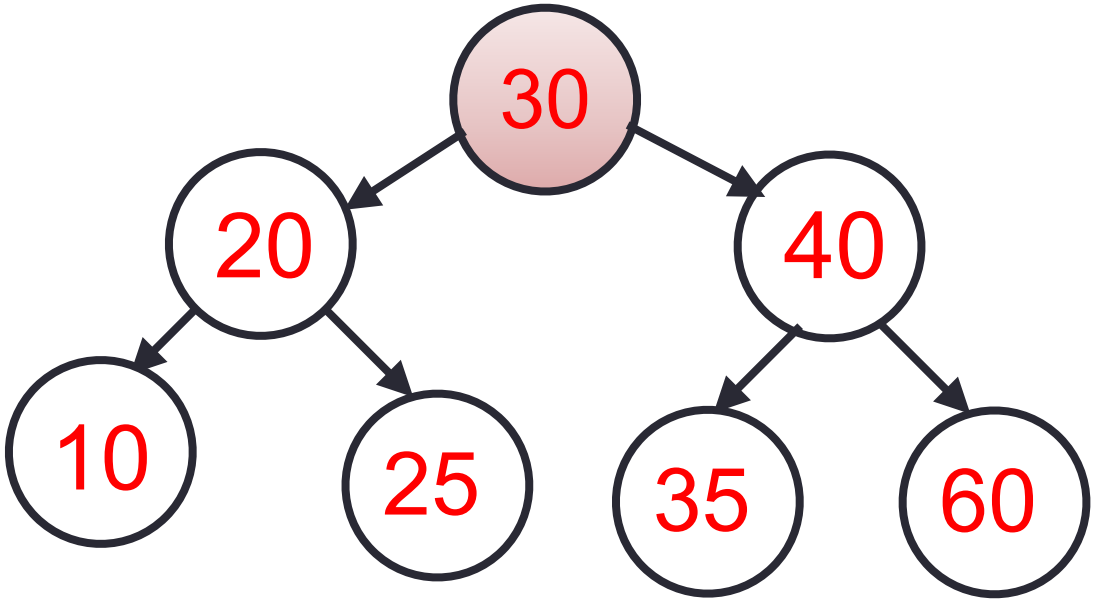
Insert keys: 41, 45, 32, 42, 12

Insert key in sorted order: 12, 32, 41, 42, 45
What is the height of the tree?
What is the best/worst case running time for insert?

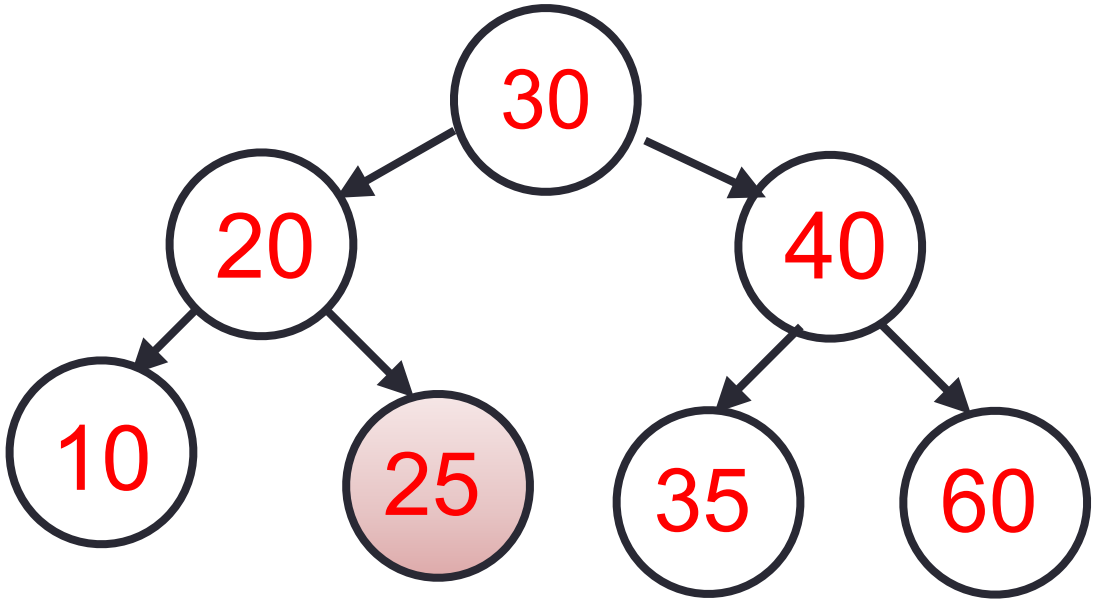
Min/Max



Successor: Next largest element

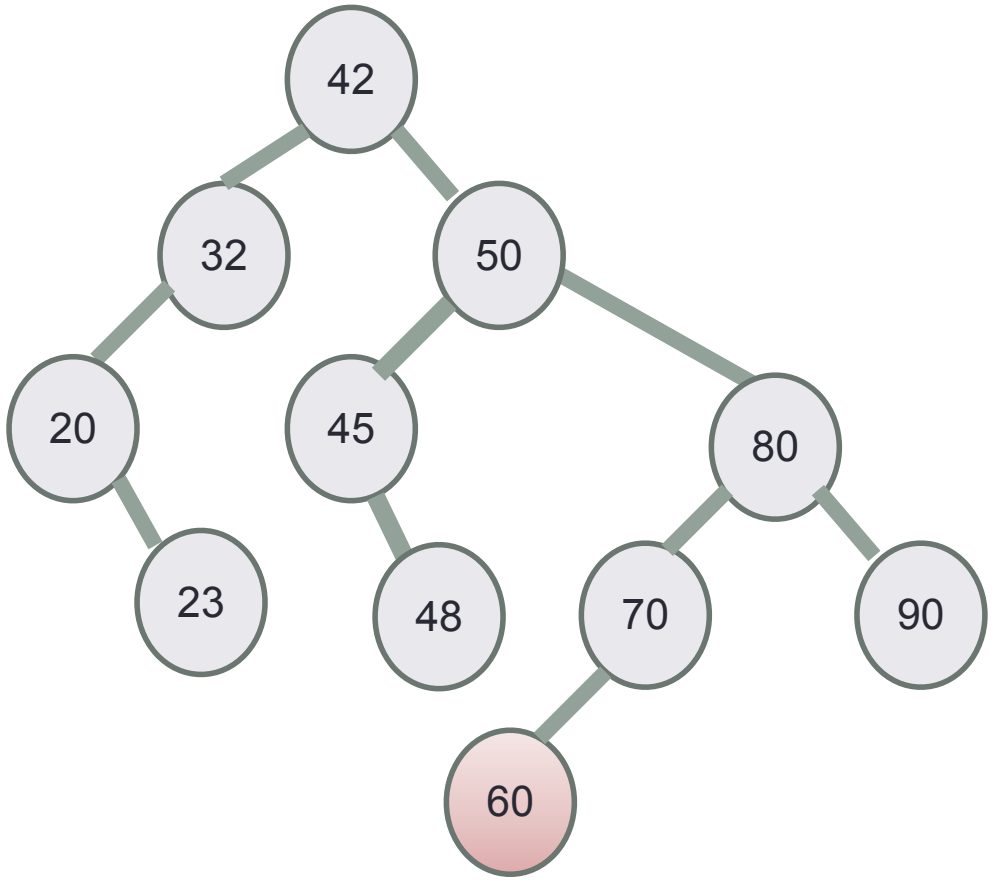


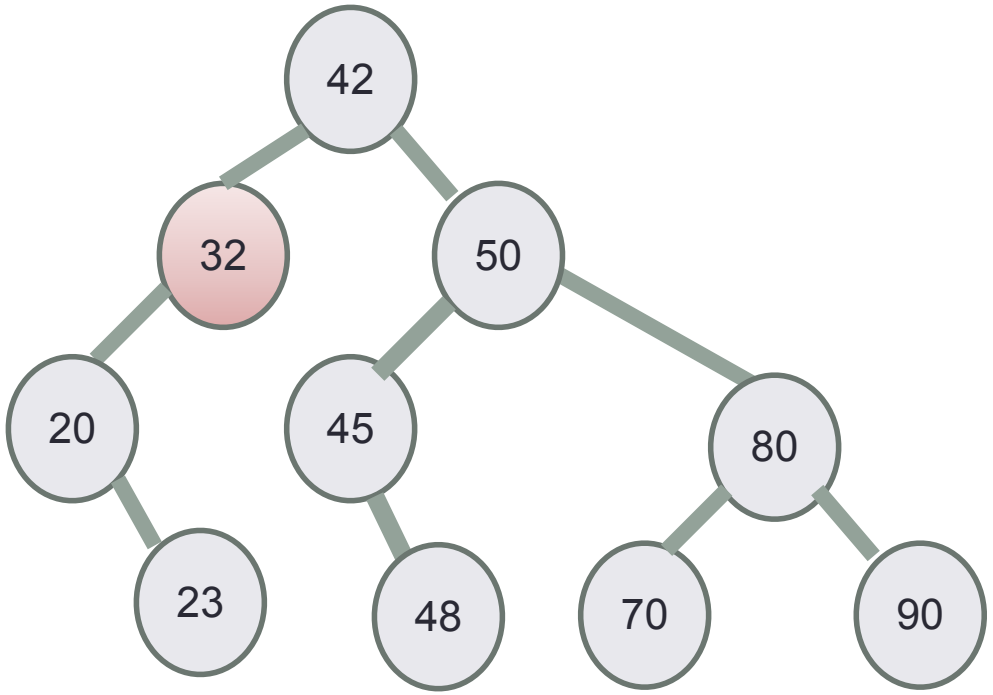
- Case 1: Node (n) has a right child



- Case 2: Node (n) has no right child

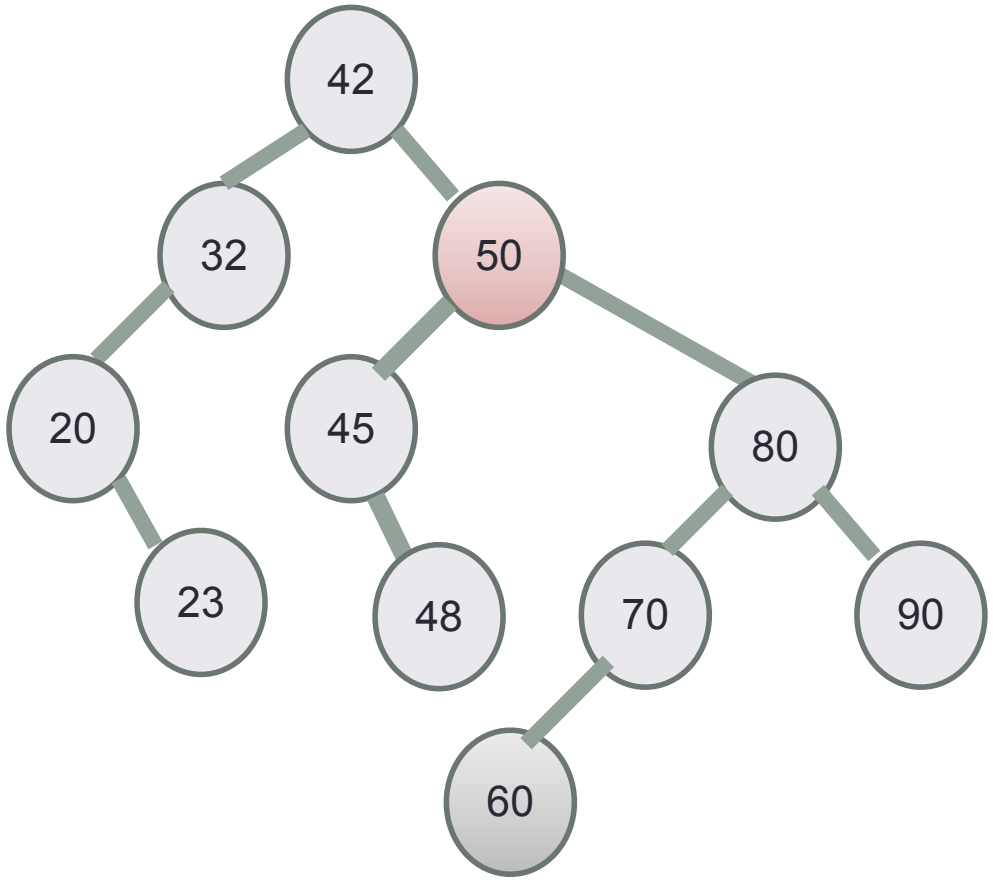
Delete a specific node



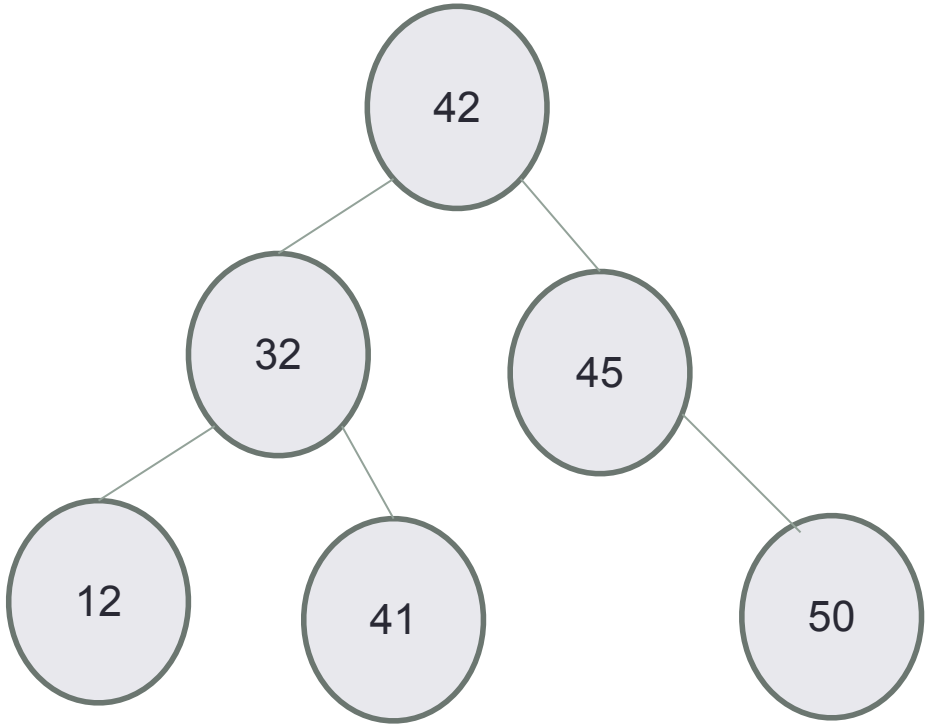


- Case 1: Node is a leaf node
  - Set parent's (left/right) child pointer to null
  - Delete the node
- Case 2: Node has one child (left/right)
  - Replace the node by its only child

Delete a specific node



- **Case 3: Node has two children**

BST ADT

Operations
Search
Insert
Min
Max
Successor
Predecessor
Delete
Print elements In order Preorder, Post order